Introduction to Programming

Week 1

Magnus Madsen

Lecture starts at 14.15 – but feel free to come down and chat

Week 1: Your first program

Monday

- Course formalities
- Software is everywhere

Thursday

- Getting started with Java
- Live programming

Prologue

Quote of the week

“To know a second language, is to have a second soul.”

— Charlemagne

Epigram of the week

“To understand a program you must become both the machine and the program.”

— Alan Perlis

Course formalities

A new course

Introduction to Programming (EN, 142222):

- A new course running for the 2nd time in fall 2026.
- Focused on the fundamentals of programming (and less focused on Java).
- All materials and lectures in English.

Staff

Lecturer: Assoc. Prof. Magnus Madsen (magnusm@cs.au.dk)

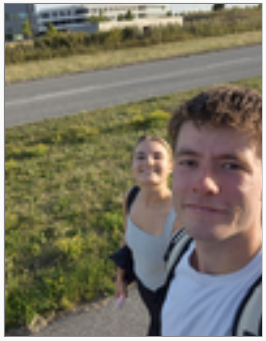
Class	Time	Room	Teaching Assistant
DA2	Tue 12–15	5523-121	Maja Landbo
DA3	Tue 12–15	5523-129	Mathias Møller Bøttger
DA4	Tue 12–15	5520-112	Tayo Kim Krüger
IT-IPROG	Wed 08–11	5520-112	Jacob Foster-Mortensen
IT-IPROG	Wed 08–11	5520-112	Noah Temkkit Gottlieb
ITE-INTPRO	Wed 12–15	5342-020	Simon Olesen
CS1-INTPRO	Wed 12–15	5125-417	Benjamín Tydor
MIX-IPROG	Wed 12–15	1520-516	Mathias Hald Kristensen
DA1	Thu 12–15	5125-120	Lasse Hjøllund Jensen

Staff

Maja

Mathias B

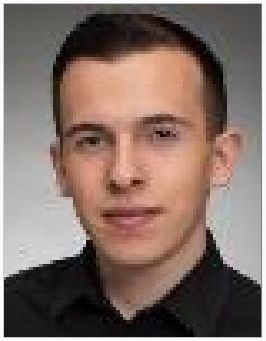
Tayo



Jacob

Noah

Simon



Benjamín

Mathias K

Lasse

Schedule

Lecture every **Monday** at **14.15-16.00** in 5335–016 (Peter Bøgh Aud.)

Lecture every **Thursday** at **08.15-10.00** in 1533–103 (Aud. E)

Deadline for mandatory assignments: every **Sunday** at **23:59**.

Mandatory assignments (1/3)

From the course description:

> **i** Students must submit a total of 10 weekly assignments which must be approved.

- There are **14** weekly assignments in total, but only **10** must be approved.
  - You may fail up to *four* assignments and still satisfy the requirement.
- The assignments are individual, but you may work together in small groups.
- The assignments have no impact on the final grade. They simply have to be approved to qualify for the exam.

Mandatory assignments (2/3)

The assignments are subject to the following requirements:

- The assignments must be submitted before the deadline: every **Sunday** at **23:59**.
  - An assignment that is not submitted before the deadline is marked FAIL.
- The TA will correct the assignment, give feedback, and mark it PASS or FAIL.
  - If an assignment is failed, you may resubmit it to address the feedback.
  - An assignment can be resubmitted once.
  - The deadline for the resubmission is *two weeks* after the original deadline.

If you cannot submit on time, you must proactively e-mail the lecturer with a reasonable explanation. TAs cannot grant extensions or exemptions.

Mandatory assignments (3/3)

Every student must receive mandatory **oral feedback** at least *four* times:

- If an assignment is marked ORAL FEEDBACK (in addition to either PASS or FAIL), you must attend the next exercise session to receive the feedback in person.
  - If you do not attend, the assignment is marked FAIL.
- The oral feedback is a conversation about the code between you and the TA.

Exam

Exam

The exam consists of two parts:

> **Part A:** An individual take-home programming project. The exam is open-book, i.e. students may use all materials available except Generative AI. Students may discuss the project, but may not share any source code.

> **Part B:** An individual written exam. The exam is closed-book, i.e. students may not use any materials. The scope of the written exam is the entire course syllabus plus the take-home programming project.

The two parts take place at different times and places.

- You have three days for the take-home programming project.
- You have two hours for the written exam.

Exam

You obtain **one grade** weighted approximately 40/60 between **Part A** and **Part B**.

The Danish grading system, the so-called “7-trins-skalaen”:

Grade	Danish	English	ECTS
12	Fremragende	Outstanding	A
10	Fortrinligt	Excellent	B
7	Godt	Good	C
4	Jævnt	OK	D
02	Tilstrækkeligt	Adequate	E
00	Utilstrækkeligt	Inadequate	Fx
-3	Ringe	Poor	F

cf. https://ufm.dk/en/education/the-danish-education-system/grading-system

How to prepare for the exam (1/2)

✓ I have read and understood the textbook and other material.

✓ I have solved all weekly exercises and understood them.

✓ I have completed all mandatory assignments and understood the TA's feedback.

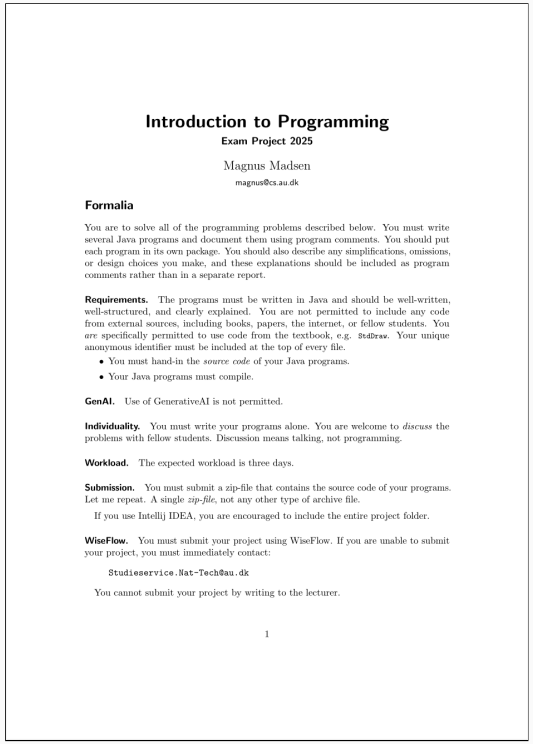
✓ I have attended lectures and exercise classes.

✓ Whenever I did not understand something, I asked for help.

If you do these things, you will be successful 🌈 🎉 🥳.

How to prepare for the exam (2/2)

Both the **exam project** and the **written exam** from 2025 are available online:



Introduction to Programming
Exam Project 2025

Magnus Madsen
magnus@cs.au.dk

Formalia

You are to solve all of the programming problems described below. You must write several Java programs and document them using program comments. You should put each program in its own package. You should also describe any simplifications, omissions, or design choices you make, and these explanations should be included as program comments rather than in a separate report.

Requirements. The programs must be written in Java and should be well-written, well-structured, and clearly explained. You are not permitted to include any code from external sources, including books, papers, the internet, or fellow students. You *are* specifically permitted to use code from the textbook, e.g. `StdDraw`. Your unique anonymous identifier must be included at the top of every file.

- You must hand-in the *source code* of your Java programs.
- Your Java programs must compile.

GenAI. Use of GenerativeAI is not permitted.

Individuality. You must write your programs alone. You are welcome to *discuss* the problems with fellow students. Discussion means talking, not programming.

Workload. The expected workload is three days.

Submission. You must submit a zip-file that contains the source code of your programs. Let me repeat. A single *zip-file*, not any other type of archive file.

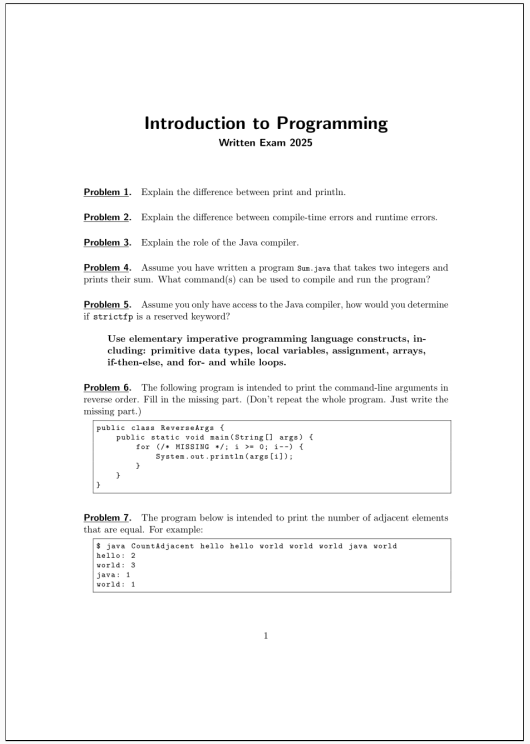
If you use Intellij IDEA, you are encouraged to include the entire project folder.

WiseFlow. You must submit your project using WiseFlow. If you are unable to submit your project, you must immediately contact:

`Studieservice.Nat-Tech@au.dk`

You cannot submit your project by writing to the lecturer.

1



Introduction to Programming
Written Exam 2025

Problem 1. Explain the difference between print and println.

Problem 2. Explain the difference between compile-time errors and runtime errors.

Problem 3. Explain the role of the Java compiler.

Problem 4. Assume you have written a program `Sum.java` that takes two integers and prints their sum. What command(s) can be used to compile and run the program?

Problem 5. Assume you only have access to the Java compiler, how would you determine if `strictfp` is a reserved keyword?

Use elementary imperative programming language constructs, including: primitive data types, local variables, assignment, arrays, if-then-else, and for- and while loops.

Problem 6. The following program is intended to print the command-line arguments in reverse order. Fill in the missing part. (Don't repeat the whole program. Just write the missing part.)

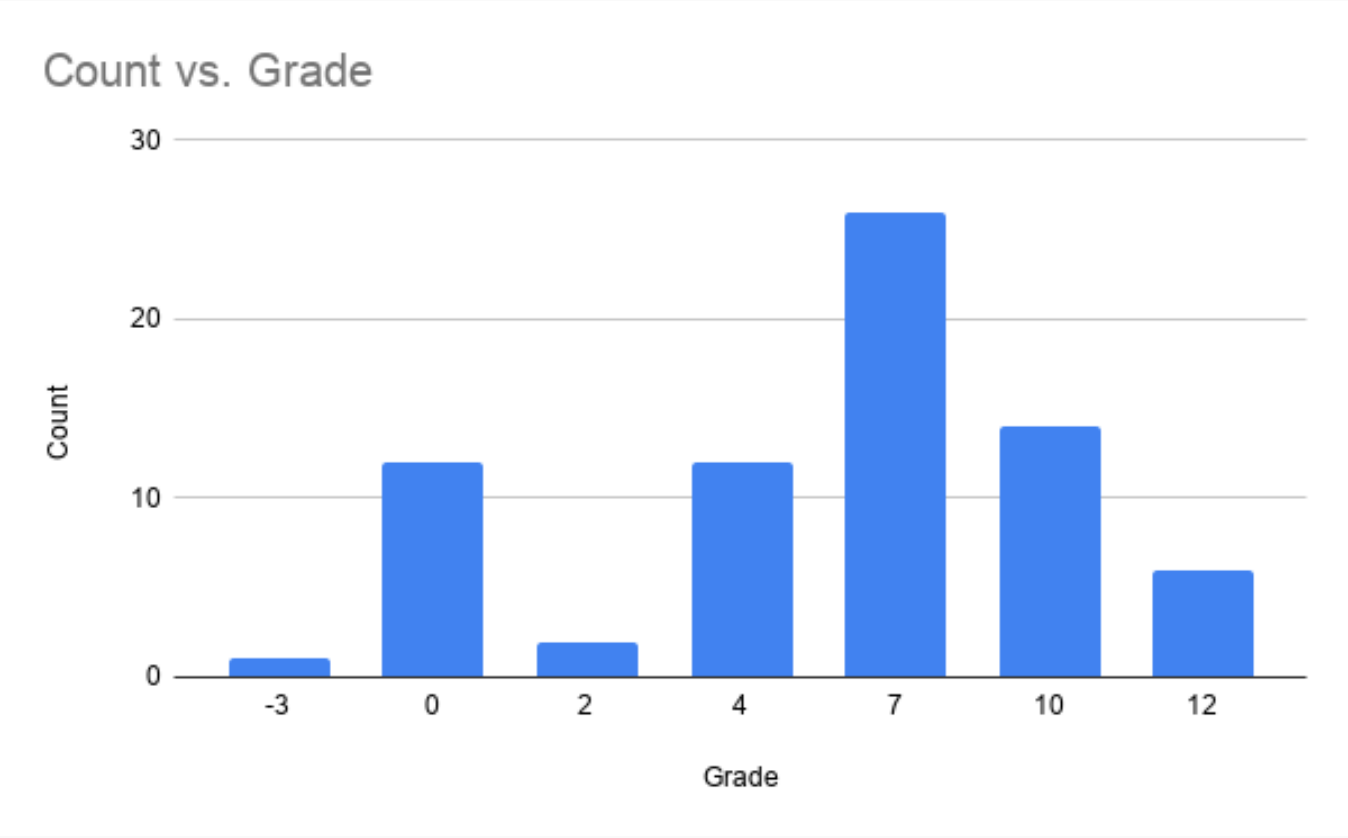
```
public class ReverseArgs {
    public static void main(String[] args) {
        for (/* MISSING */; i >= 0; i--) {
            System.out.println(args[i]);
        }
    }
}
```

Problem 7. The program below is intended to print the number of adjacent elements that are equal. For example:

```
$ java CountAdjacent hello hello world world world java world
hello: 2
world: 3
java: 1
world: 1
```

1

Grade distribution from last year



Last year, 17.8% of students received either `-3` or `00` and hence failed the exam.

Q: This is a **Q-slide**: A slide where I ask you a question and give you two minutes to discuss with the person next to you.

Q: What skills do you expect a commercial airline pilot to have?

Q: Which tasks may the pilot be unable to do without autopilot?

Generative AI (1/2)

You are probably all familiar with **large language models** (LLMs):

- **Chatbots** answer questions in conversation, e.g. ChatGPT, Copilot, Gemini, …
- **Coding agents** write and run code on your behalf, e.g. Claude Code, Codex, …

Generative AI (1/2)

You are probably all familiar with **large language models** (LLMs):

- **Chatbots** answer questions in conversation, e.g. ChatGPT, Copilot, Gemini, ...
- **Coding agents** write and run code on your behalf, e.g. Claude Code, Codex, ...

Undoubtedly, GenAI and coding agents will play a **significant role** in the future of software development.

Generative AI (1/2)

You are probably all familiar with **large language models** (LLMs):

- **Chatbots** answer questions in conversation, e.g. ChatGPT, Copilot, Gemini, ...
- **Coding agents** write and run code on your behalf, e.g. Claude Code, Codex, ...

Undoubtedly, GenAI and coding agents will play a **significant role** in the future of software development.

But...

Generative AI (2/2)

In this course, you are *not* permitted to use GenAI for assignments or for the exam.

- You have to learn the basics before you can use these tools.

You are permitted to use GenAI for your learning experience.

- You can ask for explanations of concepts.
- You can ask for hints.
- You can ask for help.

If you use them, please use them responsibly.

Freedom, responsibility, and integrity (1/3)

University is not school.

- You have a lot of freedom, but you are responsible for your own learning.

Actions that are permitted, but *not* recommended:

- Skipping lectures
- Skipping classes
- Skipping exercises
- Not reading the book

Freedom, responsibility, and integrity (2/3)

Actions with consequences:

- Not submitting the mandatory assignments
- Not submitting the mandatory assignments on time

You fail the course

- And you cannot attend the re-exam
- But you can follow the course next year
- Your diploma will not show a failure for a course you later successfully complete

Freedom, responsibility, and integrity (3/3)

Actions with severe consequences:

- Cheating
  - e.g. a friend does your assignments or exam project.
  - e.g. a friendly A.I. does your assignments or exam project.
  - ...

💀 You will be kicked out of the university 💀

- Modern AI tools watermark the text they generate[1][2]

[1] How Claude's Text Watermark Works

[2] Google DeepMind: SynthID

A good way to study

Make a **weekly plan**:

- Lectures, exercise classes, and your student job are already filled in.
- The rest is study time:
  - Allocate time for reading the book — preferably *before* the lecture.
  - Allocate time for the exercises and assignments.
  - It's tempting to start with the assignment — don't! Do a few exercises first.
  - Work together in groups – it's more fun to study together.
  - If possible, work from the university: *show up every day, read, and work*.
    - If a classmate is not there, ask why. You are a team.

When reading, focus on **understanding**: it is better to read *half* the chapter and understand it than to read the *whole* chapter and understand nothing.

If you need help, just ask. We are all here to help you learn.

Q: If you could travel back in time and give yourself one piece of advice, what would it be?

(bear with me)

A message from the future? (the past) (1/2)

“Start studying from the very beginning of the course, and do your handins properly.”

“Read more of the book throughout the semester.”

“Learn more Java specific things. General programming knowledge is not enough.”

A message from the future? (the past) (2/2)

“Do no take written part for granted.” [sic]

“Prepare for the written part and to not only learn the theory, but actually code to prepare.” [sic]

“Study a bit earlier for the written exam (I started 2 days before).”

Course description

Course content

Foundational: Understanding what a Java program is, how to compile it, and how to execute it. Reasoning about whether a program is well-formed and about its behavior.

Imperative Concepts: Writing simple methods using local variables, if-then-else, and for and while loops. Writing simple data structures using plain objects and arrays.

Object-Oriented Concepts: Writing simple classes with encapsulated state, getters and setters, and using interfaces and inheritance.

Programming Techniques: Programming with common data structures, such as lists, sets, and maps. Applying basic debugging and testing techniques to understand how a program behaves. Manipulating the file system, including the creation, reading, and writing of files.

Learning objectives (1/2)

After the course, students will be able to:

- Explain how to write a Java computer program, compile it, and execute it.
- Use elementary imperative programming language constructs, including: primitive data types, local variables, assignment, arrays, if-then-else, and for and while loops.
- Use elementary object-oriented programming language constructs, including: classes, interfaces, objects, and methods.
- Use common data structures such as lists, sets, and maps.

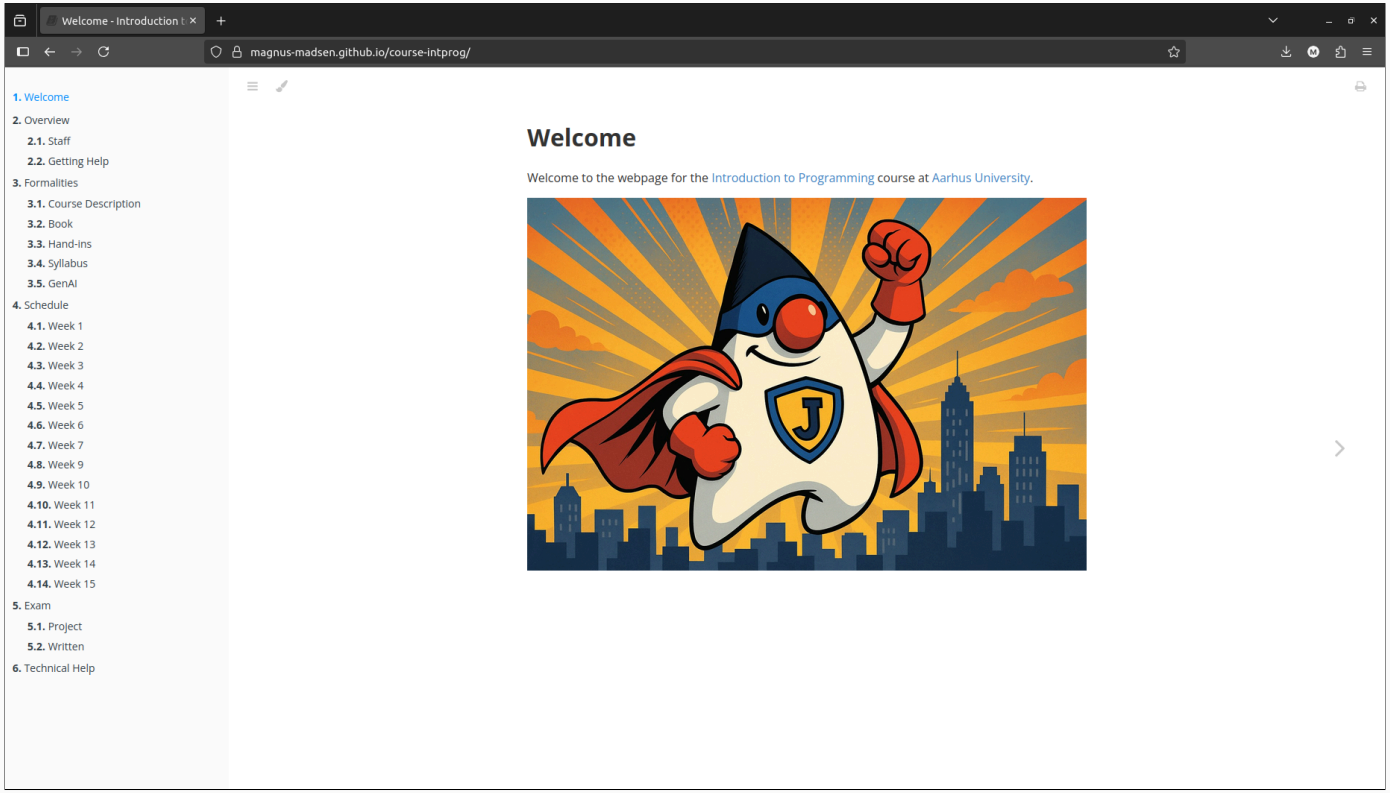
Learning objectives (2/2)

After the course, students will be able to:

- Identify, explain, and overcome compiler errors (e.g. syntax, semantic, or type errors).
- Apply programming techniques to write small programs in imperative or object-oriented style.
- Apply basic debugging and testing techniques to understand and correct program behavior.
- Apply advanced programming features such as inheritance and generics.

Learning materials

Course website



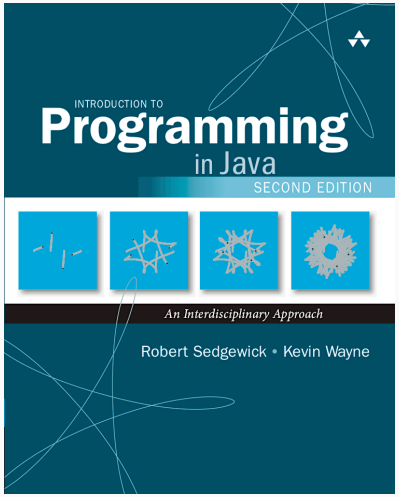
https://magnus-madsen.github.io/course-intprog/

Textbook

Introduction to Programming in Java (2nd edition).

In the words of the authors:

"... Programming is not difficult to learn and harnessing the power of the computer is rewarding ..."



❗ You must obtain the book. The exercises and assignments are in it.

❗ Ensure you get the 2nd edition. The exercises have been renumbered.

Recorded lectures

Bad news: The lectures are – by choice – not recorded 😔

We want to motivate you to participate in the lectures! 🙋

Textbook lectures

The textbook has online lectures at:

- https://www.cubits.ai/collections/106

Each lecture costs $2. You can buy all of them for $20.

Q: Is it worth it? Probably not.

Q: Do I get paid for this commercial? No.

Q: Does the internet have an excessive number of funny cat videos and Java tutorials? Yes.

How to spend your time

Total	15 hours
Lectures	4 hours
Reading	3 hours
Exercises	5 hours
Assignment	3 hours

Getting help

To get help:

- E-mail your teaching assistant for specific questions about the assignments.
- E-mail the lecturer for specific questions about the course or the exam.
- Consult “Technical Help” for help with installing Java, using IntelliJ, etc.

Programming Cafe

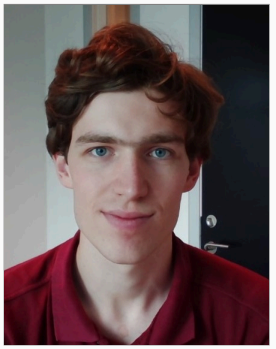
An entirely optional offer for students with *little or no* prior experience in programming.

- A three-hour session per week: **live code-along** and **group programming exercises**, with TAs there to help.
- An informal atmosphere with plenty of room for questions.
- No new material.

It is not a study cafe: the TAs here do *not* help with the exercises or mandatory assignments.

When: Monday 16–19 or Wednesday 14–17, from week 2
Where: 5522–115

Registration required.



Markus Mathiasen
markusm@cs.au.dk

Benjamin Wen
wenjamin@post.au.dk

What if you already know programming?

You probably don't know *everything*.

- Read the textbook to ensure that you understand all the details.
- Enjoy the many fun exercises in the textbook.
- While Week 1–3 may seem slow, the course rapidly picks up steam.

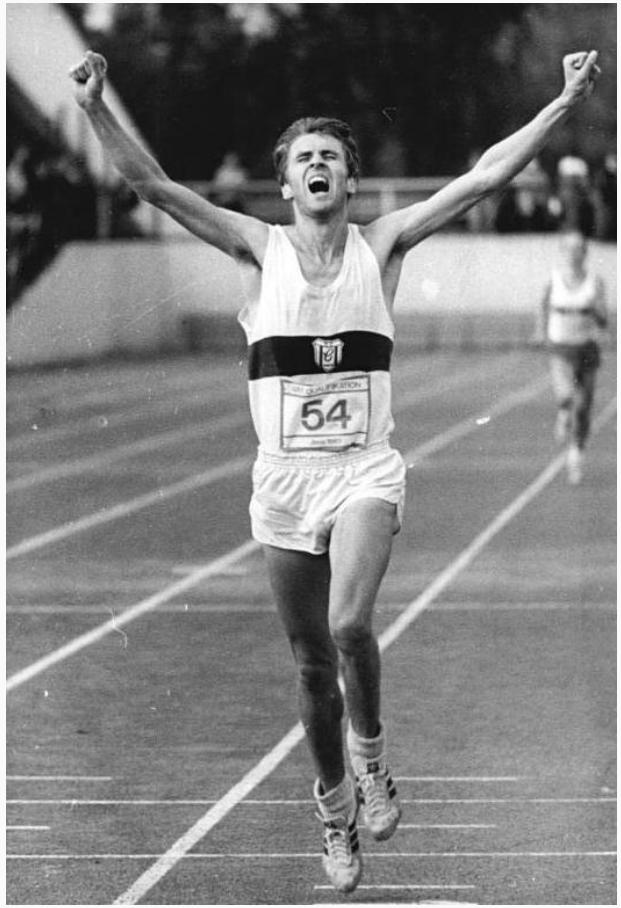
Q: Do you have any questions about the course?

How to become a programmer

Programming is a contact sport

- You cannot become a cook by reading a book.
- You cannot become a pilot by reading a book.
- You cannot become a surgeon by reading a book.
- ...

Learning by doing!



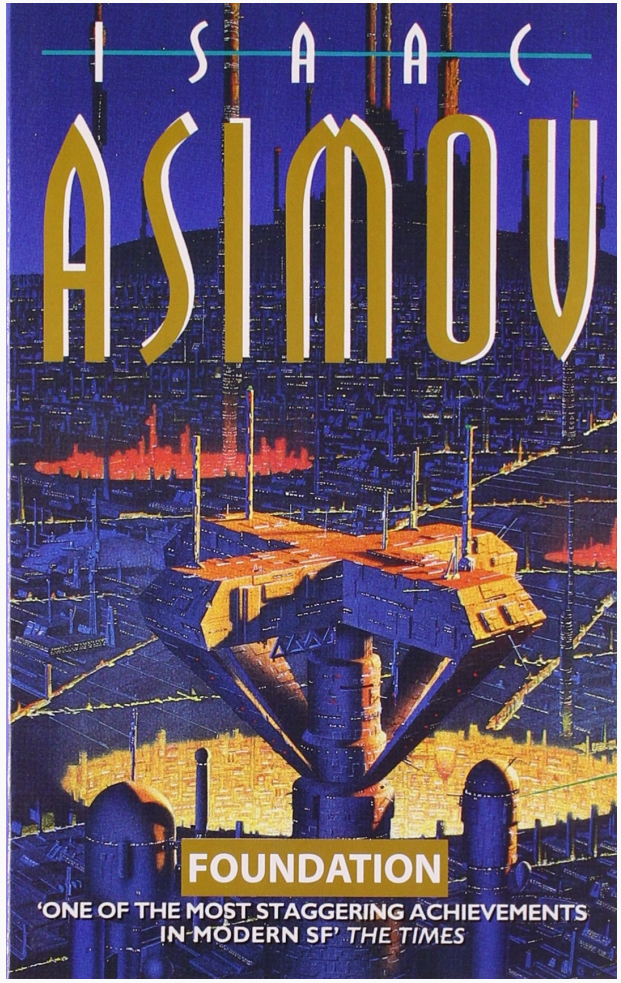
Programming is an intellectual pursuit

Programs are mathematical objects.

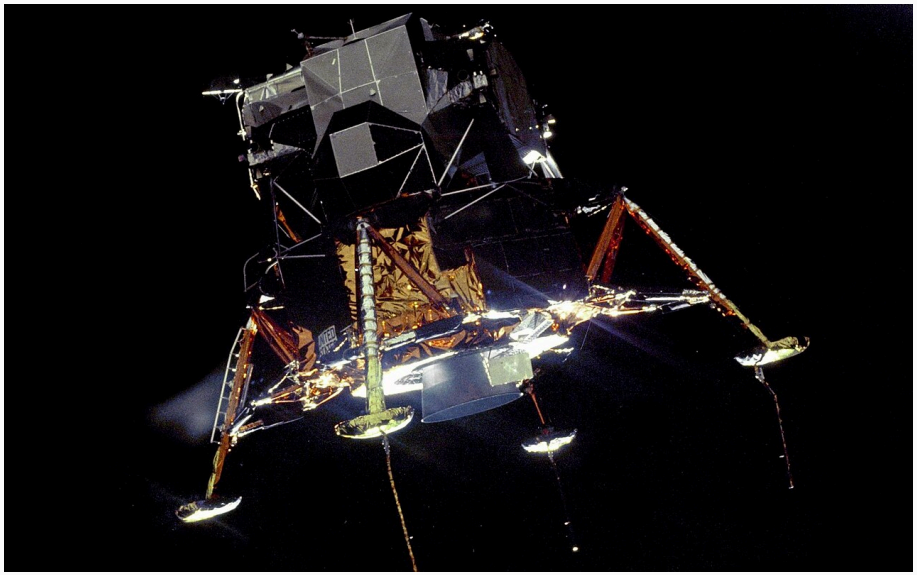
They are “mind materials” that we build with.

“To understand a program you must become both the machine and the program.” — Alan Perlis

“A mathematician is expected to sit at his computer and think.” — Isaac Asimov, Prelude to Foundation

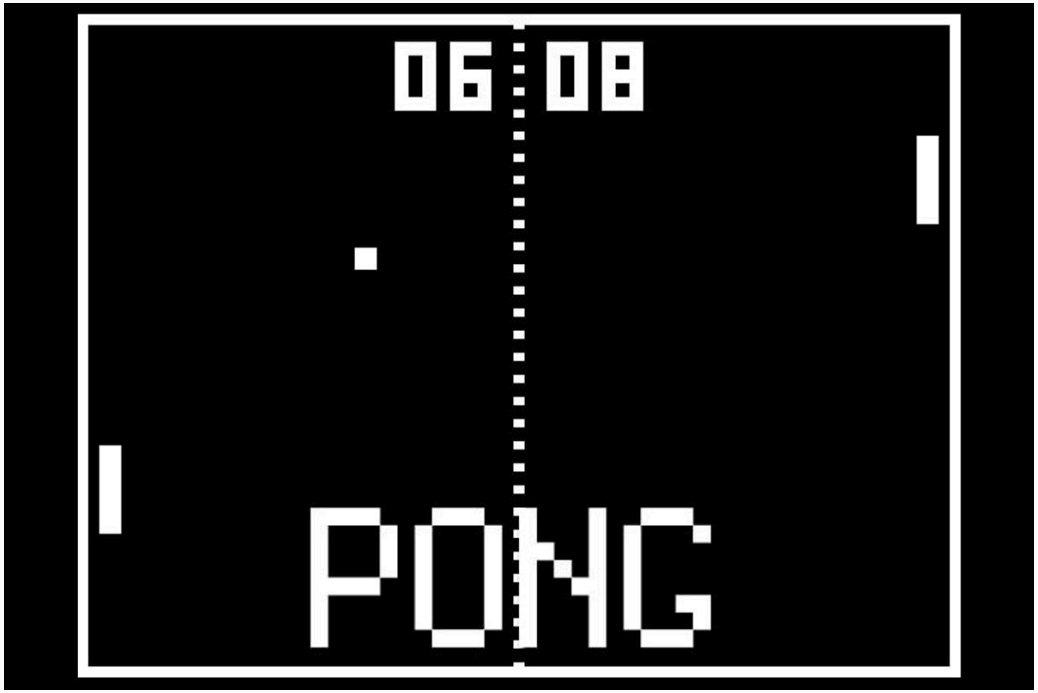


Programming is consequential



The Apollo Guidance Computer flew three people to the Moon, landed two on it, and brought them home.

Programming is fun



Programming lets you build things that are fun: games, puzzles, music, art, and entire worlds.

Software is everywhere

Software is everywhere

Q: How many computers are there in an Airbus A380?

Why study computer science?

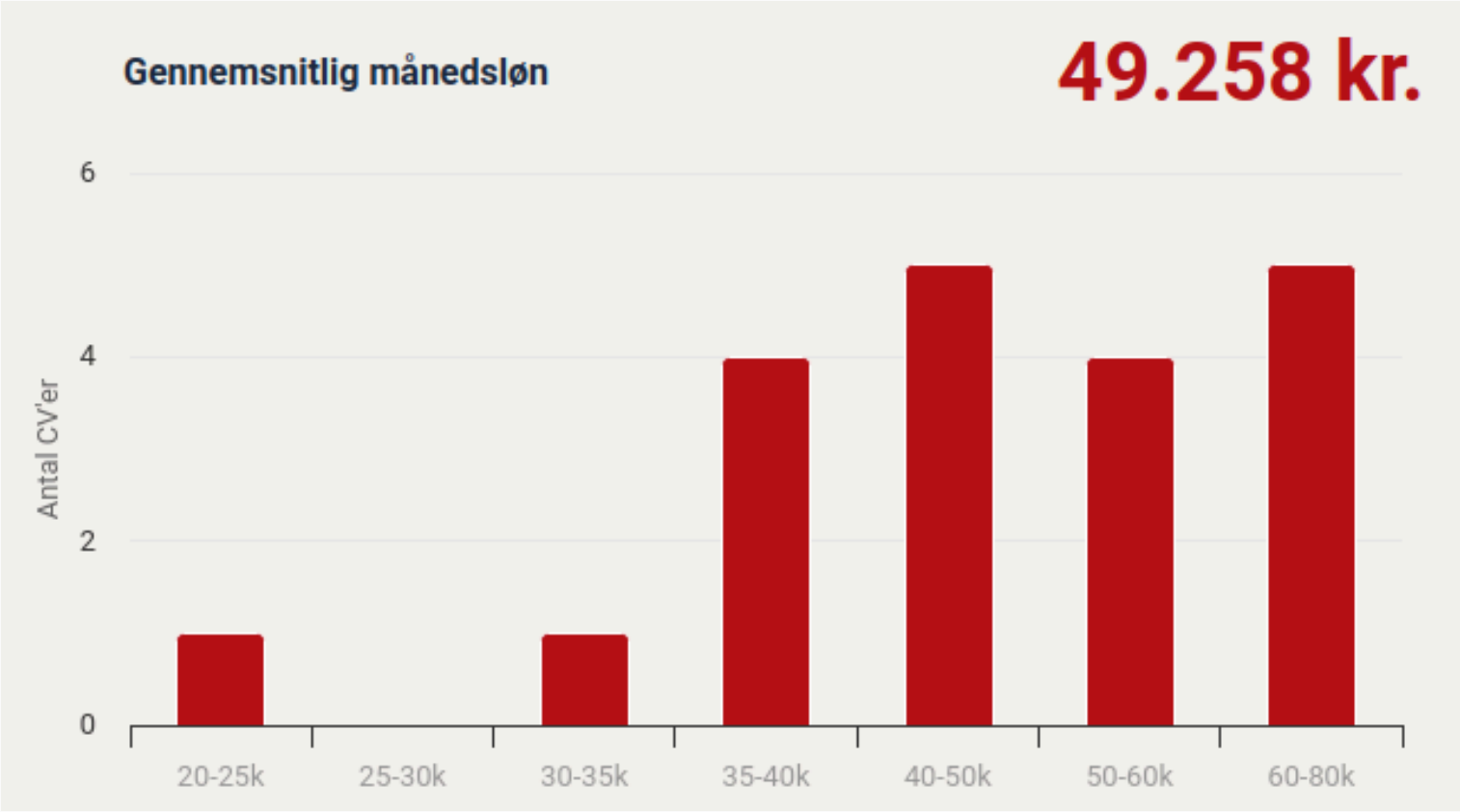
Q: Why do you want to study computer science?

Why study computer science?

Creativity	Build new things.
Problem solving	Solve difficult problems.
Impact	Build software that people use.
Flexibility	Work at a large or small company. Work on-site or from home.
Job security	Enter a field where your skills are in demand.
Salary	Earn a good salary.

Q: What is the salary of a recent computer science graduate?

Average salary



Monthly Salary

DKK	49,258
USD	7,386
EUR	6,603
JPY	1,076,233
GBP	5,562

Software engineer at Uber Aarhus with yearly bonus

Art	Specifikation			Antal	Sats	Beløb
1001	Månedsløn			160,33		96.250,00
3042	Sundhedsforsikring, beskatning			58,00		
3140	Transport Allowance					2.211,62
3191	Annual Bonus					330.000,00
8502	Løntillæg 0,45%					433,13
8750	ATP bidrag			160,33		-99,00
8861	Arbejdsmarkedsbidrag		Grundlag: 428.853,75		8,00	-34.308,00
8906	A-indk: 394.545,75	Fradrag: 0,00	Skat af 394.545,75		27,00	-106.527,00
[redacted]						-500,92
9421	ESPP B - deduction			426.250,00	15,00	-63.937,50
9993	**Overført til konto** [redacted]					**223.522,33**

Årsoplysning	År til dato	Beskrivelse	Indv. måned	År til dato
(13) AM-indkomst	617.520,75	Ferieberet. løn - kalenderår	428.952,75	578.181,75
(14) Bidragsfri A-indkomst	0,00	Ferieberet. løn - ferieår	428.952,75	956.285,70
(15) A-skat	153.391,00	Fritvalgskonto	0,00	0,00
(16) AM-bidrag	49.400,00			

DKK	428,853
USD	35,117
EUR	31,300
JPY	5,099,344
GBP	26,340

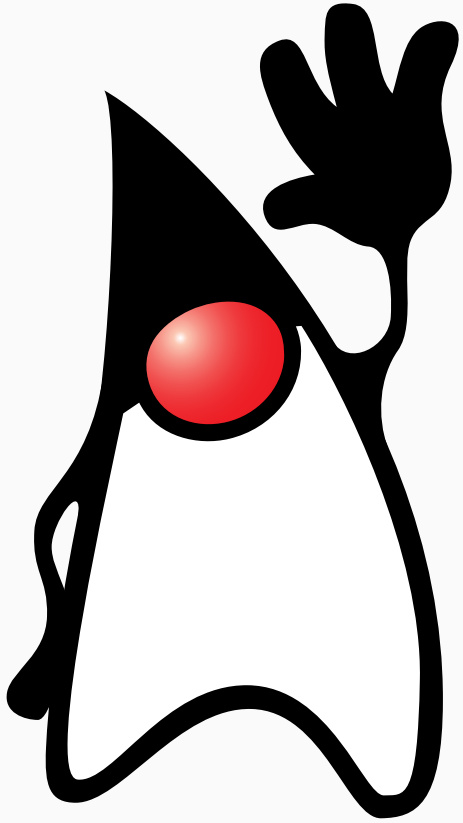
Summary

- Submit the **mandatory assignments** on time: every **Sunday** at **23:59**.
- Programming is **learning by doing**.

Getting started with Java

What is Java?

- A programming language
- A compiler (`javac`)
- A runtime (`java`)
- A standard library
- ...



Why Java?

Java

- has a simple and readable syntax that is easy to learn.
- has a standard library with a lot of built-in functionality.
- has a static type system that catches errors before your program runs.
- has extensive IDE and tooling support.

Java ...

- is a **general-purpose** language useful for any type of project.
- is widely adopted with over 16 million projects on GitHub.
- is open source and freely available.

Java ...

- programs run reasonably fast (faster than programs in most other languages).
- programs run on any platform, including Windows, macOS, and Linux.

Why not Java?

Java ...

- is verbose and requires more boilerplate code than modern languages.
- was designed in the computing era of the 1990s for single-core desktop systems.

Java ...

- lacks support for functional programming compared to modern languages.
- lacks resource and memory safety features found in languages such as Rust.
- faces competition from modern alternatives like Kotlin, Rust, and Scala.

Upshot: You will learn many programming languages over your career.

Your first program

We can write a program:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World");
    }
}
```

and save it in the file `HelloWorld.java`.

Compiling a program (1/2)

We compile the program by opening a terminal and running the command:

```
$ javac HelloWorld.java
```

Surprisingly, if the program compiles successfully, there is no output.

Running the `javac` command produces a `HelloWorld.class` file.

Compiling a program (2/2)

The `HelloWorld.java` file is a **text file**, but the `HelloWorld.class` file is a **binary file**:

```
0000000 feca beba 0000 4100 1d00 000a 0002 0703
0000010 0400 000c 0005 0106 1000 616a 6176 6c2f
0000020 6e61 2f67 624f 656a 7463 0001 3c06 6e69
0000030 7469 013e 0300 2928 0956 0800 0900 0007
0000040 0c0a 0b00 0c00 0001 6a10 7661 2f61 616c
0000050 676e 532f 7379 6574 016d 0300 756f 0174
0000060 1500 6a4c 7661 2f61 6f69 502f 6972 746e
0000070 7453 6572 6d61 083b 0e00 0001 480c 6c65
0000080 6f6c 202c 6f57 6c72 0a64 1000 1100 0007
0000090 0c12 1300 1400 0001 6a13 7661 2f61 6f69
00000a0 502f 6972 746e 7453 6572 6d61 0001 7007
```

Decompiling

We can inspect the binary file with `javap`:

```
$ javap -c HelloWorld.class
```

```
public class HelloWorld {
  /* ... */
  public static void main(java.lang.String[]);
    Code:
       0: getstatic     #7  // Field java/lang/System.out
       3: ldc           #13 // String Hello, World
       5: invokevirtual #15 // Method java/io/PrintStream.println
       8: return
}
```

Run a program

After compiling a program, we can run it by invoking the `java` command.

```
$ java HelloWorld
```

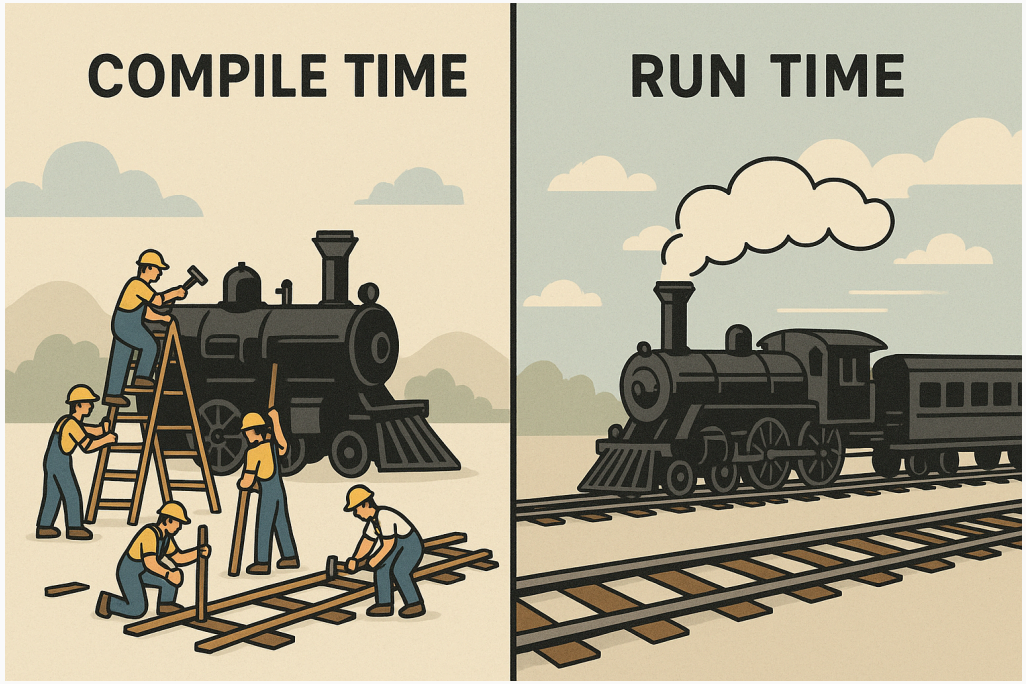
The terminal responds with:

```
Hello, World
```

Compile time vs. run time

Compile time is when your program is under construction. You can get a compile-time error when your program is not *valid Java code*.

Run time is when your program is *running*. You can get a runtime error when your program does not do what you want.



Command line arguments

Command line arguments allow the user to specify an *input* to the program.

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello " + args[0] + "!");
    }
}
```

Compile once	`$ javac Main.java`		
Execute with many inputs	`$ java Main Alice`	`$ java Main Bob`	`$ java Main World`
	`Hello Alice!`	`Hello Bob!`	`Hello World!`

Programming tools

A programmer's toolkit consists of many specialized tools that help development:

Essential Tools:

- Command Line Interface (CLI)
- Compilers
- Interpreters
- Text Editors

Advanced Tools:

- Integrated Development Environments (IDEs)
- Version Control Systems (VCS)
- Debuggers
- Profilers
- Coding Agents

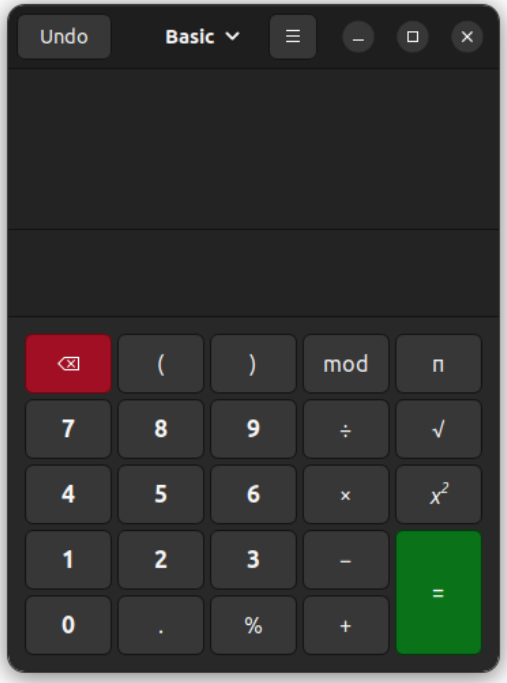
> **i** Each tool serves a specific purpose in the software development lifecycle. We will explore these in detail on the following slides.

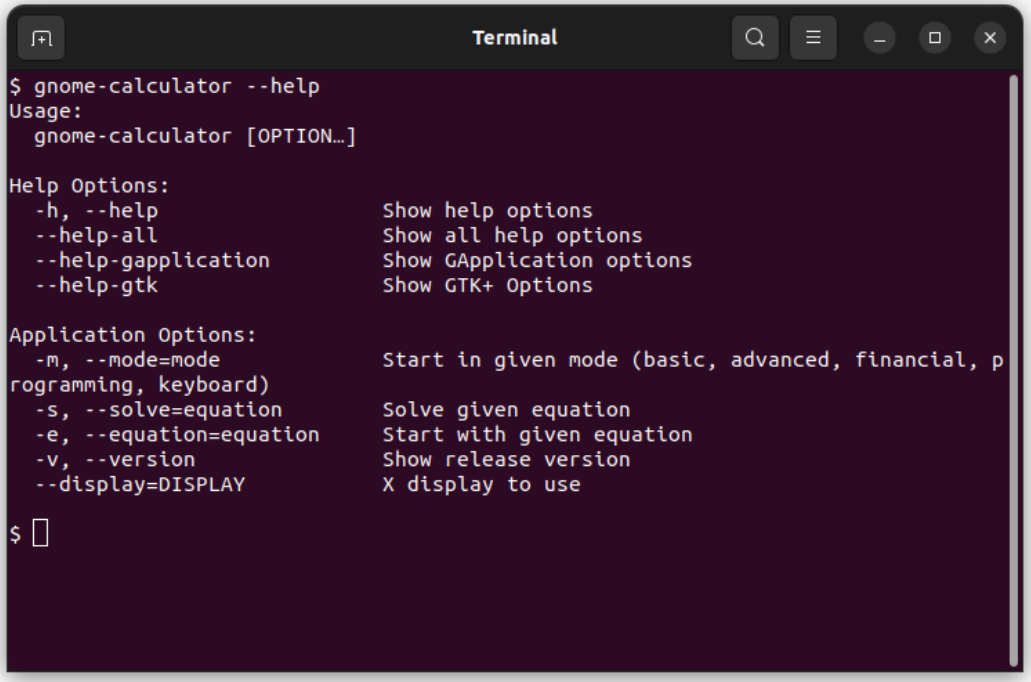
Q: *Raise your hand:* How many of you are familiar with working in the terminal (using the CLI)?

Command-line interface (1/2)

You may be used to interacting with programs via a **Graphical User Interface** (GUI).

The **Command Line Interface** (CLI) is more **powerful** and more **automatable**.





Command-line interface (2/2)

With the CLI we can combine commands into **pipelines** to answer questions about our data.

Suppose the file `grades.txt` contains one student per line:

```
$ cat grades.txt
thyra  PASS
gorm   FAIL
dagmar PASS
holger PASS
```

Command-line interface (2/2)

With the CLI we can combine commands into **pipelines** to answer questions about our data.

Suppose the file `grades.txt` contains one student per line:

```
$ cat grades.txt
thyra  PASS
gorm   FAIL
dagmar PASS
holger PASS
```

We can compute an alphabetical list of the students who passed:

```
$ cat grades.txt | grep PASS | sort
dagmar PASS
holger PASS
thyra  PASS
```

Interpreters

Interpreters read source code and execute it **directly**, line by line, without creating a separate executable file.

How Interpreters Work:

- Read source code directly
- Parse and execute instructions immediately
- No separate compilation step required
- Code runs as soon as you type it

Examples:

- Python (`python script.py`)
- JavaScript (in web browsers)
- Ruby (`ruby script.rb`)
- Shell scripts (`bash script.sh`)

Advantages:

✓ Fast development cycle
✓ Interactive testing (REPL)
✓ Cross-platform portability
✓ Easy debugging

Disadvantages:

❌ Slower execution speed
❌ Runtime error discovery
❌ Requires interpreter

Compilers

Compilers translate code from one language to another.

Usually this is from a high-level human-readable language (such as Java) to a low-level computer-readable language (such as **bytecode** or **assembly**).

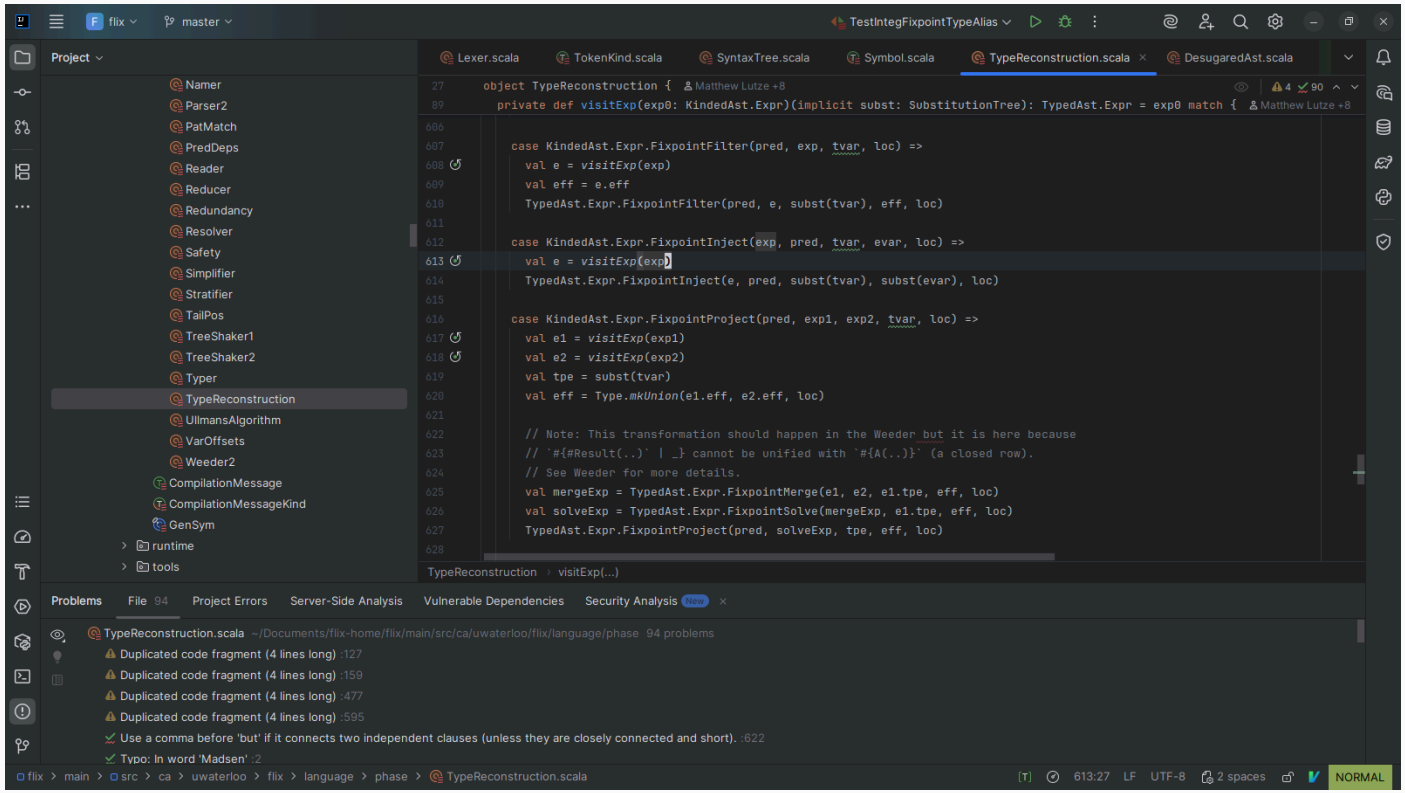
```
public class MyProgram {
    public static void main(String[] args)
        { }
}
```

```
0000000 feca beba 0000
0000010 0400 000c 0005
0000020 6e61 2f67 624f
0000030 7469 013e 0300
0000040 0c0a 0b00 0c00
0000050 676e 532f 7379
0000060 1500 6a4c 7661
```

The low-level language is typically faster for a computer to execute.

Integrated Development Environments (IDEs)

An **Integrated Development Environment** is like Microsoft Word for programmers.

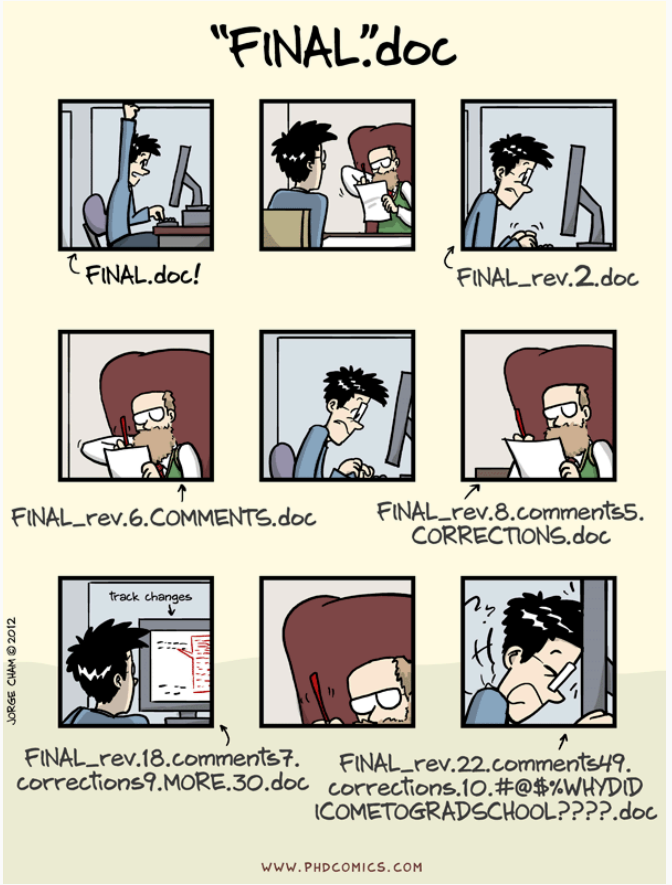


Programs are just text, but IDEs help you write, organize, test, and run your code.

Version Control Systems (VCSs)

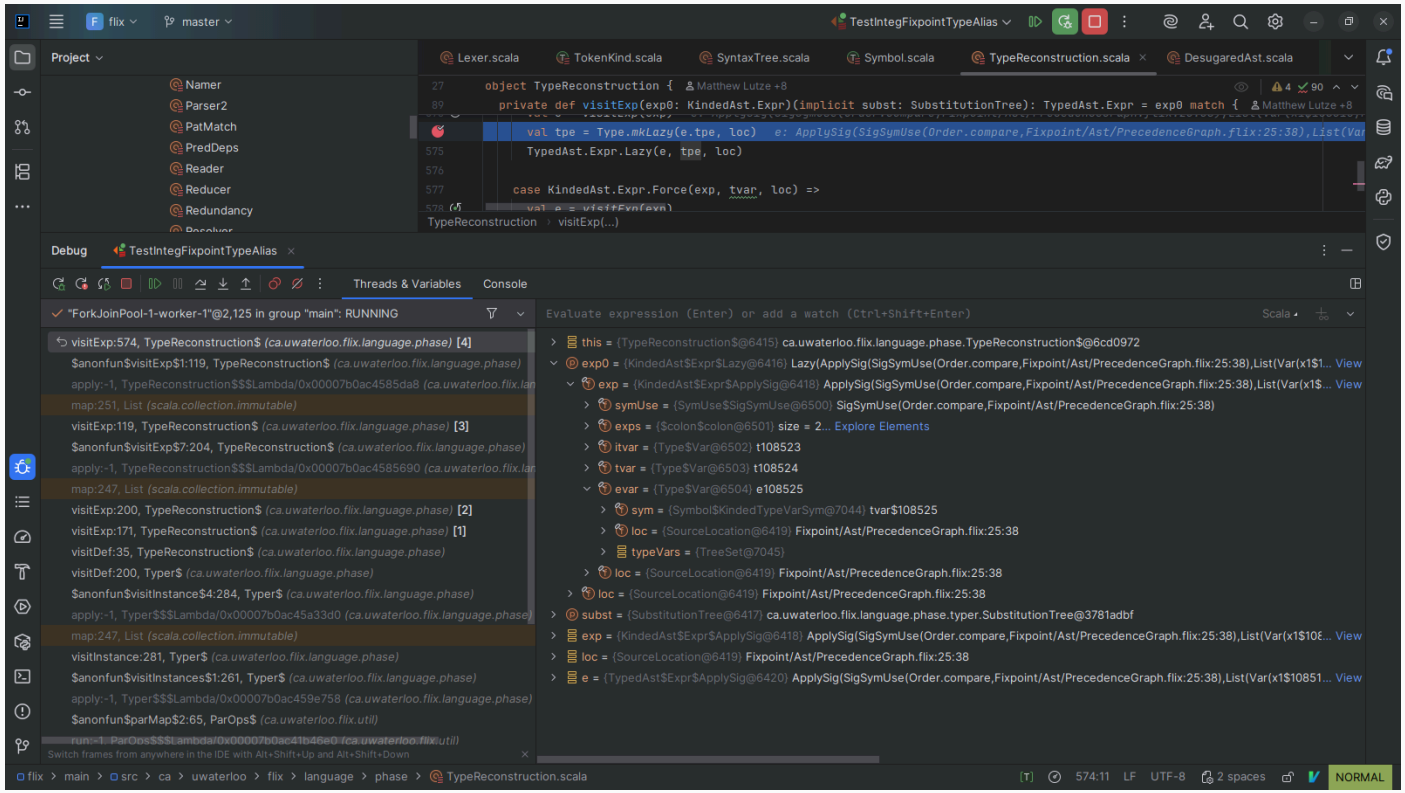
Version Control Systems, such as **Git**, let the programmer manage *versions* of a project.

- They track the complete history of your code changes over time.
- They make it possible to collaborate with others on the same project.



Debuggers

Debuggers allow us to inspect the **state** of a program while it is running.



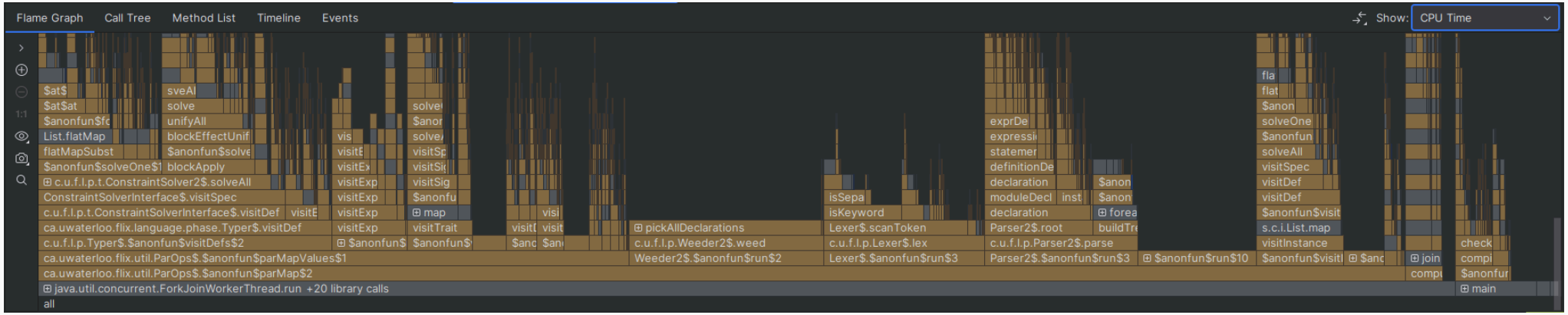
They are often integrated into an IDE.

Profilers

Profilers track the **time** and **memory** usage of our code.

We can use them to identify and fix bottlenecks – places in our code that run slowly.

For example, a **flame graph** shows the time spent in each part of a program.



Epilogue

Error of the week (1/2)

What is the problem here?

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
```

Error of the week (1/2)

What is the problem here?

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
```

The Java compiler reports:

```
Main.java:4: error: reached end of file while parsing
```

Do you understand the issue now?

Error of the week (2/2)

What is the problem here?

```
$ javac MyProgram.java
$ java MyProgram.class
```

Error of the week (2/2)

What is the problem here?

```
$ javac MyProgram.java
$ java MyProgram.class
```

The terminal reports:

```
Error: Could not find or load main class MyProgram.class
Caused by: java.lang.ClassNotFoundException: MyProgram.class
```

Do you understand the issue now?

Live programming

Live programming

Today, we shall see how to:

- compile and run a program with nothing but a text editor, `javac`, and `java`.
  - open the terminal.
  - check if Java is correctly installed.
- use IntelliJ IDEA to write a program and run it from the terminal.
- use IntelliJ IDEA to write a program and run it from within IDEA.
- have multiple programs in the same IDEA project.

Sources for images and slides

- https://introcs.cs.princeton.edu/java/lectures/
- https://commons.wikimedia.org/wiki/File:Bundesarchiv_Bild_183-1983-0528-011,_Werner_Schildhauer.jpg
- https://jeroenthoughts.wordpress.com/wp-content/uploads/2019/01/foundation1.jpg
- https://commons.wikimedia.org/wiki/File:Apollo_11_Mission_Control_Center_-_landing_trajectory.jpg
- https://commons.wikimedia.org/wiki/File:Apollo_11_Lunar_Module_Eagle_in_landing_configuration_in_lunar_orbit_from_the_Command_and_Service_Module_Columbia.jpg
- https://cdn.i-scmp.com/sites/default/files/styles/1200x800/public/2015/06/05/pong.jpg
- https://commons.wikimedia.org/wiki/File:NASA_Mars_Rover.jpg
- https://commons.wikimedia.org/wiki/File:Surgeon_performing_robot-assisted_surgery.jpg
- https://commons.wikimedia.org/wiki/File:Airbus_A380_cockpit.jpg
- https://commons.wikimedia.org/wiki/File:%E7%96%AF%E7%8B%82%E7%9C%BC%E9%95%9C%E8%9B%87%E8%BF%87%E5%B1%B1%E8%BD%A6.JPG
- https://commons.wikimedia.org/wiki/File:Daft_Punk_in_2013_2.jpg
- https://commons.wikimedia.org/wiki/File:Claas_Lexion_480-20080806-RM-133439.jpg
- https://www.reddit.com/r/dkloenseddel/comments/1k0l761/softwareingeni%C3%B8r_aarhus/
- https://commons.wikimedia.org/wiki/File:Duke_(Java_mascot)_waving.svg
- https://phdcomics.com/comics/archive.php?comicid=1531